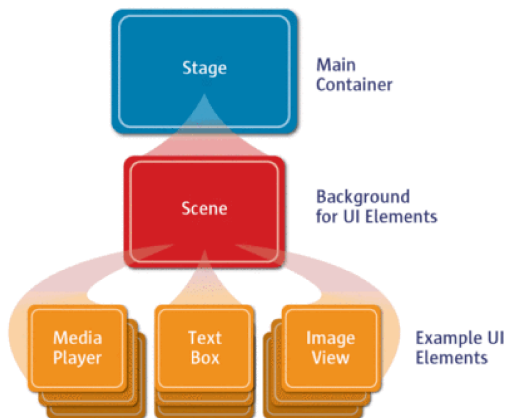


JavaFX Cheatsheet

Ecco il testo ripulito dagli a capo superflui (rispettando tabelle e blocchi di codice) e con i tag dei code block corretti da `Java` a `java` :

Struttura base



Ogni applicazione JavaFX deve estendere `Application` e sovrascrivere il metodo `start(Stage stage)` .

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.scene.layout.FlowPane;
4  import javafx.stage.Stage;
5
6  public class EsempioBase extends Application {
7      @Override
8      public void start(Stage stage) {
9          stage.setTitle("Titolo finestra");
10
11         // 1. Creare il pannello (layout)
12         FlowPane root = new FlowPane();
13
14         // 2. Creare i componenti (Label, Button, ...)
15
16         // 3. Aggiungere componenti al pannello
17         // root.getChildren().addAll(comp1, comp2, ...);
18
19         // 4. Creare la scena (larghezza/altezza opzionali)
20         Scene scene = new Scene(root, 400, 300);
21
22         // 5. Impostare la scena e mostrare
23         stage.setScene(scene);
24         stage.show();
25     }
26
27     public static void main(String[] args) {
28         launch(args);
29     }
30 }
```

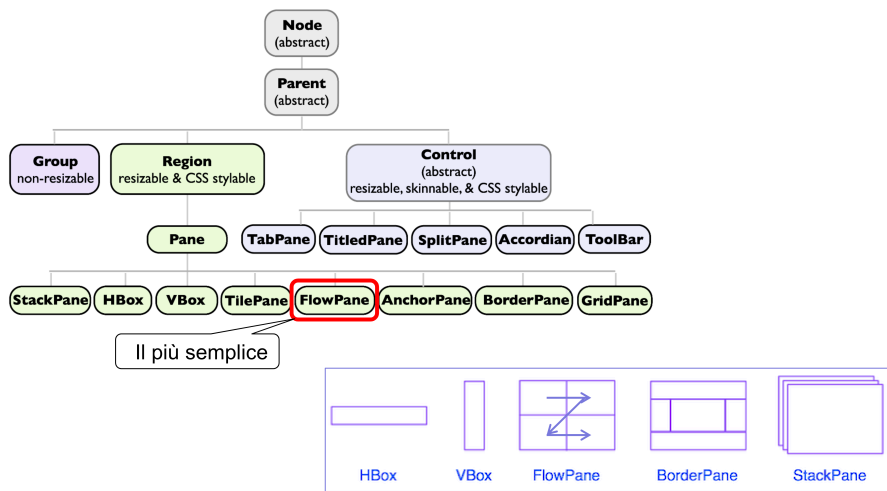
Metodo `start` : è il punto di ingresso della parte grafica.

`Stage` : la finestra principale.

`Scene` : contenitore che associa il pannello radice (root) alla finestra.

Layout Panes

I pannelli dispongono automaticamente i componenti figli secondo una politica. **Non usare coordinate assolute.**



Es.



Pannello	Comportamento	Uso tipico
FlowPane	Dispone in riga, va a capo quando non c'è spazio	Pannello generico, esercizi semplici
VBox	Disposizione verticale (una colonna)	Moduli, liste verticali
HBox	Disposizione orizzontale (una riga)	Barra pulsanti
BorderPane	5 aree: top, bottom, left, right, center	Interfacce con menu, barra di stato
GridPane	Griglia righe/colonne	Form complessi
StackPane	Componenti sovrapposti (uno sopra l'altro)	Immagini di sfondo

Esempio con BorderPane:

```

1  BorderPane root = new BorderPane();
2  root.setTop(new Label("Intestazione"));
3  root.setCenter(lista);
4  root.setBottom(new Button("OK"));

```

Esempio con VBox:

```

1  VBox vbox = new VBox(10); // spaziatura verticale 10 pixel
2  vbox.getChildren().addAll(label, textField, button);

```

Metodi comuni a tutti i pannelli (Region/Pane)

Metodo	Descrizione
getChildren().addAll(...)	Aggiunge uno o più nodi figli al pannello
getChildren().clear()	Rimuove tutti i nodi figli
setStyle(String)	Imposta il colore di sfondo o altro via CSS (es. "-fx-background-color: lightgray;")
setPadding(Insets)	Margine interno (es. new Insets(10))
setSpacing(double)	Distanza tra componenti (solo VBox, HBox, FlowPane)
setAlignment(Pos)	Allinea il contenuto (es. Pos.CENTER)

Gestione degli eventi – tutti i modi

Un evento è un'istanza di una sottoclasse di `EventObject` . Per i pulsanti e toggle, l'evento più comune è `ActionEvent` .

Per ascoltare un evento, si usa un `EventHandler` (interfaccia funzionale con un solo metodo `handle`).

Lambda expression (modo più usato)

```
1 button.setOnAction(event -> {
2     System.out.println("Cliccato");
3     // event.getSource() restituisce il componente che ha generato l'evento
4 });
```

Method reference

```
1 button.setOnAction(this::gestisciClic);
2
3 private void gestisciClic(ActionEvent event) {
4     // codice
5 }
```

Classe esterna separata

```
1 class MioListener implements EventHandler<ActionEvent> {
2     private Label label;
3     public MioListener(Label lbl) { this.label = lbl; }
4     public void handle(ActionEvent e) {
5         label.setText("Cliccato");
6     }
7 }
8 // Uso:
9 button.setOnAction(new MioListener(myLabel));
```

ChangeListener (per proprietà osservabili)

Usato per slider, valueProperty, selectedItemProperty, ecc.

```
1 slider.valueProperty().addListener((observable, oldValue, newValue) -> {
2     // observable è la proprietà, oldValue e newValue sono i valori
3     double corrente = newValue.doubleValue();
4     //chiama myHandle?
5 });
```

La lambda ha 3 parametri.

Attenzione: non tutti i componenti generano `ActionEvent` . Lo Slider non lo genera, serve `ChangeListener` sulla proprietà `valueProperty` .

Componente Label

Ruolo: visualizzare testo statico o dinamico (solo output, non interattivo).

Costruttori

```
1 Label l1 = new Label(); // vuoto
2 Label l2 = new Label("Testo iniziale"); // con testo
3 Label l3 = new Label("Testo", imageView); // con immagine/icona
```

Metodo	Descrizione
<code>setText(String)</code>	Cambia il testo visualizzato
<code>getText()</code>	Restituisce il testo corrente
<code>setFont(Font font)</code>	Imposta carattere, dimensione, stile
<code>setTextFill(Paint)</code>	Colore del testo (es. <code>Color.RED</code>)
<code>setStyle(String)</code>	CSS diretto (es. <code>"-fx-font-size: 20px;"</code>)
<code>setWrapText(boolean)</code>	Se <code>true</code> , manda a capo il testo automaticamente
<code>setGraphic(Node)</code>	Inserisce un elemento grafico (es. icona) di fianco al testo
<code>setAlignment(Pos)</code>	Allinea il testo all'interno dell'area della Label

Font

```
1 // Font normale
```

```

2  label.setFont(Font.font("Arial", 16));
3
4  // Grassetto
5  label.setFont(Font.font("Courier New", FontWeight.BOLD, 24));
6
7  // Corsivo
8  label.setFont(Font.font("Times New Roman", FontPosture.ITALIC, 18));

```

Componente Button

Ruolo: pulsante che genera `ActionEvent` al clic. Non mantiene lo stato.

Costruttori

```

1  Button b1 = new Button();           // senza testo
2  Button b2 = new Button("Cliccami"); // con testo
3  Button b3 = new Button("", new ImageView(...)); // con icona

```

Metodo	Descrizione
<code>setText(String)</code>	Cambia l'etichetta
<code>getText()</code>	Legge l'etichetta attuale
<code>setOnAction(EventHandler)</code>	Definisce l'azione da eseguire al clic
<code>setToolTip(Tooltip)</code>	Aggiunge un suggerimento al passaggio del mouse
<code>setDisable(boolean)</code>	Se <code>true</code> , disabilita il pulsante
<code>setDefaultButton(boolean)</code>	Se <code>true</code> , si attiva premendo INVIO
<code>setCancelButton(boolean)</code>	Se <code>true</code> , si attiva premendo ESC
<code>setGraphic(Node)</code>	Inserisce un'icona (es. <code>ImageView</code>)
<code>fire()</code>	Simula il clic via codice

```

1  button.setOnAction(event -> {
2      // lambda expression
3  });

```

Componenti di testo

Metodi condivisi (da `TextInputControl`)

Questi metodi valgono per `TextField`, `PasswordField` e `TextArea`:

Metodo	Descrizione
<code>getText()</code>	Legge il testo contenuto
<code>setText(String)</code>	Imposta/sovrascrive il testo
<code>clear()</code>	Cancella tutto il contenuto
<code>setEditable(boolean)</code>	Rende modificabile (<code>true</code>) o in sola lettura (<code>false</code>)
<code>setPromptText(String)</code>	Mostra un testo sfumato se il campo è vuoto
<code>appendText(String)</code>	Aggiunge testo in fondo senza sostituire il precedente
<code>selectAll()</code>	Seleziona tutto il testo

TextField (riga singola)

Costruttori:

```

1  TextField tf = new TextField();           // vuoto
2  TextField tf2 = new TextField("Testo iniziale"); // con testo

```

Metodo specifico	Descrizione
<code>setPrefColumnCount(int)</code>	Imposta la larghezza preferita in caratteri
<code>setOnAction(EventHandler)</code>	Gestisce l'evento pressione tasto INVIO nel campo

PasswordField (come `TextField` ma nasconde i caratteri)

```

1  PasswordField pf = new PasswordField();

```

```

2  pf.setPromptText("Inserisci password");
3  String pwd = pf.getText(); // restituisce la stringa effettiva decodificata

```

TextArea (multi-riga)

Costruttori:

```

1  TextArea area = new TextArea();
2  TextArea area2 = new TextArea("Testo\nsu più righe");

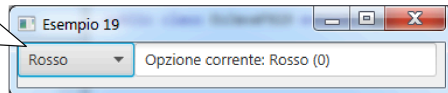
```

Metodo specifico	Descrizione
setPrefRowCount(int)	Numero di righe visibili
setPrefColumnCount(int)	Larghezza in caratteri
setWrapText(boolean)	Se true, va a capo in automatico senza tagliare le parole

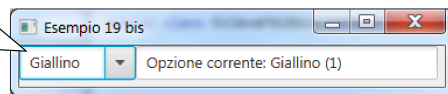
Componente ComboBox

Ruolo: lista a discesa. Di default non editabile; se editabile l'utente può scrivere un valore personalizzato.

ComboBox non editabile: funziona come una **ChoiceBox**



ComboBox editabile: si possono scrivere valori non previsti in elenco



```

1  ComboBox<String> combo = new ComboBox<>();

```

Metodo	Descrizione
getValue()	Restituisce l'elemento selezionato (o null se nessuno)
setValue(T)	Imposta la selezione a un elemento esistente
getItems()	Restituisce la <code>ObservableList</code> per usare <code>.add()</code> / <code>.addAll()</code>
setItems(ObservableList)	Sostituisce la lista degli elementi
getSelectionModel().getSelectedIndex()	Indice selezionato (0-based)
getSelectionModel().clearSelection()	Deseleziona tutto
setEditable(boolean)	Permette all'utente di scrivere testo libero
setPromptText(String)	Testo mostrato quando nessuna selezione
setPrefWidth(double)	Larghezza preferita
setOnAction(EventHandler)	Evento scatenato al cambio di selezione

ComboBox editabile

```

1  combo.setEditable(true);
2  combo.setPromptText("Scegli o scrivi");
3  // Il testo scritto è accessibile con combo.getEditor().getText()
4  // ma attenzione: getValue() restituisce l'elemento selezionato dalla lista,
5  // non il testo libero.

```

Componente Slider

Ruolo: barra di scorrimento per selezionare un valore numerico in un intervallo.

Costruttori

```

1  Slider s1 = new Slider(); // min=0, max=100, valore=0
2  Slider s2 = new Slider(0, 100, 50); // min, max, valore iniziale

```

Metodo	Descrizione
setMin(double) / setMax(double)	Imposta i limiti inferiori e superiori
setValue(double)	Imposta valore corrente
getValue()	Restituisce il valore corrente (double)

Metodo	Descrizione
valueProperty()	Proprietà osservabile (per il <code>ChangeListener</code>)
setOrientation(Orientation)	Es. <code>Orientation.VERTICAL</code> (default orizzontale)
setShowTickMarks(boolean)	Mostra i segni di spunta visivi
setShowTickLabels(boolean)	Mostra le etichette numeriche
setMajorTickUnit(double)	Distanza tra i tick principali
setMinorTickCount(int)	Numero di tick piccoli tra due tick grandi
setSnapToTicks(boolean)	Forza il cursore ad allinearsi esattamente sui tick
setBlockIncrement(double)	Incremento/decremento tramite frecce tastiera

Componente ToggleButton

Ruolo: pulsante a due stati (selezionato / deselezionato). Mantiene lo stato dopo il clic.

Costruttore

```
1  ToggleButton tb = new ToggleButton("On/Off");
```

Metodo	Descrizione
isSelected()	Restituisce <code>true</code> se il pulsante è nello stato premuto (ON)
setSelected(boolean)	Forza programmaticamente lo stato del pulsante
setToggleGroup(ToggleGroup)	Associa il pulsante a un gruppo (per opzioni mutuamente esclusive)
setOnAction(EventHandler)	Azione eseguita quando il pulsante viene cliccato

Componente CheckBox

Ruolo: casella di opzione (selezionata/non selezionata). Ogni `CheckBox` è indipendente.

Costruttore

```
1  CheckBox chk = new CheckBox("Accetto i termini");
```

Metodo	Descrizione
isSelected()	Restituisce <code>true</code> se la casella è spuntata
setSelected(boolean)	Imposta lo stato di spunta
setAllowIndeterminate(boolean)	Consente uno stato "terzo" (indeterminato/trattino)
setOnAction(EventHandler)	Azione eseguita al cambio di stato

Componente RadioButton con ToggleGroup

Ruolo: opzioni mutuamente esclusive (una sola per gruppo). I `RadioButton` vanno sempre raggruppati.

Creazione del gruppo

```
1  ToggleGroup gruppo = new ToggleGroup();
```

Metodo di RadioButton	Descrizione
setToggleGroup(ToggleGroup)	Associa il <code>RadioButton</code> al gruppo (fondamentale!)
isSelected()	Verifica se è il <code>RadioButton</code> correntemente selezionato
setSelected(boolean)	Seleziona forzatamente il <code>RadioButton</code>

Metodo di ToggleGroup	Descrizione
getSelectedToggle()	Restituisce il <code>Toggle</code> (<code>RadioButton</code>) attualmente selezionato
selectedToggleProperty()	Proprietà osservabile per usare il <code>ChangeListener</code>
getToggles()	Restituisce la lista di tutti i bottoni nel gruppo

Componente ListView

Ruolo: lista verticale con scrollbar. Permette selezione singola o multipla.

Popolamento

```
1  ListView<String> list = new ListView<>();
2  ObservableList<String> items = FXCollections.observableArrayList(
3      "Mela", "Pera", "Banana"
4  );
5  list.setItems(items);
```

Metodo	Descrizione
<code>setItems(ObservableList)</code>	Popola la lista con gli elementi
<code>getItems()</code>	Restituisce gli elementi (per <code>.add()</code> , <code>.remove()</code>)
<code>getSelectionModel().getSelectedItem()</code>	Restituisce l'elemento selezionato
<code>getSelectionModel().getSelectedItems()</code>	Restituisce la lista degli elementi selezionati (in selezione multipla)
<code>getSelectionModel().setSelectionMode()</code>	Imposta <code>SelectionMode.SINGLE</code> o <code>MULTIPLE</code>
<code>getSelectionModel().clearSelection()</code>	Deseleziona tutto
<code>setPrefHeight(double)</code>	Altezza preferita in pixel
<code>setPrefWidth(double)</code>	Larghezza preferita in pixel

Componente ChoiceBox

Ruolo: simile a `ListView` ma mostra gli elementi in una tendina che si apre al clic. Occupa meno spazio.

Metodo	Descrizione
<code>getValue()</code>	Restituisce l'elemento attualmente selezionato
<code>setValue(T)</code>	Imposta programmaticamente l'elemento selezionato
<code>setItems(ObservableList)</code>	Popola la tendina
<code>getItems()</code>	Permette di accedere e modificare gli elementi
<code>setPrefWidth(double)</code>	Larghezza della tendina
<code>setOnAction(EventHandler)</code>	Listener chiamato quando si sceglie un valore

Differenza con `ComboBox`: `ChoiceBox` non è editabile (non si può scrivere). `ComboBox` sì, con `setEditable(true)` .

Componente Spinner

Ruolo: campo con frecce su/giù per selezionare un valore numerico o da una lista.

Per numeri interi

```
1  Spinner<Integer> spinner = new Spinner<>(min, max, valoreIniziale);
2  // Esempio: da 0 a 10, parte da 5
3  Spinner<Integer> spin = new Spinner<>(0, 10, 5);
```

Metodo	Descrizione
<code>getValue()</code>	Restituisce il valore corrente
<code>getValueFactory().setValue(T)</code>	Imposta forzatamente un nuovo valore
<code>valueProperty()</code>	Proprietà osservabile per il <code>ChangeListener</code>
<code>increment()</code> / <code>increment(int)</code>	Aumenta il valore di uno step (o <i>n</i> step) da codice
<code>decrement()</code> / <code>decrement(int)</code>	Diminuisce il valore di uno step (o <i>n</i> step) da codice
<code>setEditable(boolean)</code>	Permette all'utente di digitare il numero direttamente

Componenti `ProgressBar` e `ProgressIndicator`

Ruolo: mostrare l'avanzamento di un'operazione (0.0 = 0%, 1.0 = 100%).

Costruzione

```
1  ProgressBar bar = new ProgressBar(0.25); // 25%
2  ProgressIndicator indicator = new ProgressIndicator(0.25);
```

Metodo	Descrizione
<code>setProgress(double)</code>	Imposta l'avanzamento (da 0.0 a 1.0). Se passiamo <code>-1.0</code> , diventa indeterminata (animazione continua)

Metodo	Descrizione
<code>getProgress()</code>	Legge l'avanzamento corrente
<code>progressProperty()</code>	Proprietà per il binding o <code>ChangeListener</code>

Componente DatePicker

Ruolo: selezione data tramite calendario pop-up.

Costruzione

```
1 DatePicker datePicker = new DatePicker();
```

Metodo	Descrizione
<code>getValue()</code>	Restituisce la data selezionata come <code>LocalDate</code> (può essere <code>null</code>)
<code>setValue(LocalDate)</code>	Imposta programmaticamente la data
<code>setPromptText(String)</code>	Mostra un testo di suggerimento se vuoto
<code>setShowWeekNumbers(boolean)</code>	Mostra i numeri della settimana nel calendario popup
<code>setEditable(boolean)</code>	Permette la digitazione testuale della data

Componente ColorPicker

Ruolo: selezionare un colore tramite una tavolozza.

Costruzione

```
1 ColorPicker colorPicker = new ColorPicker();
```

Metodo	Descrizione
<code>getValue()</code>	Restituisce un oggetto di tipo <code>Color</code>
<code>setValue(Color)</code>	Imposta un colore di default
<code>getCustomColors()</code>	Restituisce/modifica la lista dei colori custom salvati dall'utente
<code>setOnAction(EventHandler)</code>	Evento lanciato quando si sceglie il colore

Componente ImageView

Ruolo: visualizzare un'immagine.

Costruzione

```
1 Image image = new Image("file:alberi.jpg"); // da file
2 ImageView imageView = new ImageView(image);
```

Metodo (ImageView)	Descrizione
<code>setImage(Image)</code>	Sostituisce l'immagine visualizzata
<code>getImage()</code>	Restituisce l'oggetto <code>Image</code> caricato
<code>setFitWidth(double)</code>	Ridimensiona alla larghezza desiderata
<code>setFitHeight(double)</code>	Ridimensiona all'altezza desiderata
<code>setPreserveRatio(boolean)</code>	Se <code>true</code> , mantiene le proporzioni originali durante il resize
<code>setSmooth(boolean)</code>	Applica un filtro di qualità al resize dell'immagine

Dialogo FileChooser

Ruolo: finestra modale per aprire o salvare file.

Creazione

```
1 FileChooser chooser = new FileChooser();
2 chooser.setTitle("Seleziona un file");
```

Metodo	Descrizione
<code>setTitle(String)</code>	Imposta il titolo della finestra di dialogo

Metodo	Descrizione
<code>setInitialDirectory(File)</code>	Apri il FileChooser in una cartella specifica di partenza
<code>getExtensionFilters()</code>	Restituisce la lista a cui fare <code>.add()</code> per i filtri di formato
<code>showOpenDialog(Window)</code>	Apri la finestra di selezione (singola) e restituisce un <code>File</code>
<code>showSaveDialog(Window)</code>	Apri la finestra di salvataggio e restituisce un <code>File</code>
<code>showOpenMultipleDialog(...)</code>	Restituisce una <code>List<File></code> permettendo la selezione multipla

Grafica a pixel con Canvas

Un componente `Canvas` funge da vera e propria tela sulla quale poter disegnare grafici a pixel. Come qualsiasi altro elemento grafico di JavaFX, può essere aggiunto ai figli del pannello principale.

Per iniziare a tracciare forme, linee o testi, dal `Canvas` si deve recuperare l'entità denominata contesto grafico (`GraphicsContext`). Questo oggetto offre tutti i metodi operativi necessari, ad esempio quelli contrassegnati dal prefisso `strokeXXX` (per disegnare contorni vuoti) e `fillXXX` (per disegnare forme piene).

Creazione del Canvas ed esempi base:

```

1  Canvas canvas = new Canvas(150, 130);
2  panel.getChildren().add(canvas); // Aggiunta del Canvas alla lista dei figli del pannello [cite: 6, 8]
3
4  GraphicsContext g = canvas.getGraphicsContext2D(); // Recupero dell'entità che serve per disegnare [cite: 9, 12, 27]
5
6  // Uso dei metodi grafici del GraphicsContext [cite: 10, 30]
7  // Lo specifico font si imposta tramite la factory internalizzata Font.font [cite: 29]
8  g.setFont(Font.font("Serif", FontWeight.BOLD, 20));
9  g.setFill(Color.RED); // Colore figure piene [cite: 30, 32]
10 g.fillRect(20, 20, 100, 80); // Rettangolo pieno [cite: 30]
11
12 g.setFill(Color.BLUE);
13 g.strokeRect(30, 30, 80, 60); // Rettangolo (solo contorno) [cite: 30]
14 g.fillText("ciao", 50, 60); // Testo sul canvas [cite: 30]
```

Disegnare grafici di funzione su Canvas

Per effettuare il grafico di una funzione (ad esempio sinusoidale o fratta), poiché nell'API base non esiste un oggetto di tipo `FloatUnaryOperator`, si utilizza l'interfaccia funzionale generica `Function<Float, Float>`. Tramite il metodo `apply` si calcola programmaticamente la coordinata Y (es. `float y = f.apply(x);`).

Occorre in seguito implementare un algoritmo di trasformazione delle coordinate (da logiche della funzione a fisiche in pixel del display), applicando dei fattori di scala calcolati rispetto alle dimensioni del `Canvas` e ai limiti degli assi `xAxisMin`, `xAxisMax`, `yAxisMin` e `yAxisMax` preimpostati.

```

1  // Variabili d'appoggio per calcolare i fattori di scala e convertire la funzione logica in pixel fisici
2  float fattoreDiScalax = larghezza / ((float) xAxisMax - xAxisMin); [cite: 46]
3  float fattoreDiScalay = altezza / ((float) yAxisMax - yAxisMin); [cite: 46]
4
5  // 1. Scorrimento della X sui pixel della tela del canvas [cite: 75]
6  for (int ix = 1; ix < g.getCanvas().getWidth(); ix++) {
7      // Trasformazione del pixel (ix) nella coordinata X logica
8      float x = xAxisMin + ((float) ix) / fattoreDiScalax; [cite: 75]
9
10     // Calcolo della corrispondente Y chiamando la lambda function
11     float y = f.apply(x);
12
13     // 2. Passaggio delle coordinate alla funzione di utilità custom (es: setPixel o setLine) [cite: 81]
14     setLine(g, xPrev, yPrev, x, y);
15     xPrev = x;
16     yPrev = y; [cite: 81]
17 }
```

Regole e calcoli nella stampa dei punti:

All'interno dei metodi personalizzati (`setPixel` e `setLine`), il passaggio chiave è il calcolo degli indici matematici finali.

- L'indice x: `int ix = Math.round((x - xAxisMin) * fattoreDiScalax);`
- L'indice y (sottratto all'altezza totale per l'inversione dell'asse Y classico dei display): `int iy = altezza - Math.round((y - yAxisMin) * fattoreDiScalay);`

Un punto cruciale per prevenire anomalie visive è aggiungere un controllo all'inizio di queste routine per accertarsi che le coordinate restino entro i margini grafici (es. verificando che `x < xAxisMin || x > xAxisMax || y < yAxisMin || y > yAxisMax` generino un `return`). Questo accorgimento è di vitale importanza soprattutto quando si affronta il grafico di funzioni discontinue; se omettiamo i controlli (ed impieghiamo la

funzione `setLine()`, JavaFX finirà per collegare visivamente i salti della funzione originando falsi segmenti verticali. Infine, per accendere fisicamente un singolo pixel dopo averne tracciato le coordinate logiche si usa `g.strokeLine(ix, iy, ix, iy);` (per un punto) oppure `g.strokeLine(ixP, iyP, ix, iy);` (per collegare due punti continui).

Schema generale per i grafici XY

Tutti i grafici basati su assi (`BarChart`, `LineChart`, `AreaChart`, `ScatterChart`, `BubbleChart`) seguono gli stessi passi.

Passi da compiere

1. **Predisporre gli assi** – di solito un `CategoryAxis` (categorie testuali) e un `NumberAxis` (valori numerici). Per `BubbleChart` servono due `NumberAxis` (x e y entrambi numerici).
2. **Creare l'oggetto Chart** – es. `new BarChart<>(asseX, asseY)`. Impostare il titolo con `setTitle(String)`.
3. **Predisporre le serie di dati** – una o più istanze di `XYChart.Series<X, Y>`.
4. **Popolare ciascuna serie** – aggiungere `new XYChart.Data<>(xValue, yValue)` alla serie.
5. **Aggiungere le serie al grafico** – `chart.getData().addAll(serie1, serie2, ...)`.

Tipi di assi

Asse	Tipo	Uso
<code>CategoryAxis</code>	valori testuali (categorie)	per l'asse orizzontale di <code>BarChart</code> , <code>LineChart</code> , <code>AreaChart</code> , <code>ScatterChart</code>
<code>NumberAxis</code>	valori numerici	per l'asse verticale (o per entrambi in <code>BubbleChart</code>)

Componente BarChart

Grafico a barre: una barra per ogni coppia (categoria, valore). Più serie affiancano le barre.

Esempio – una sola serie (vendite di frutta a Modena)

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.scene.chart.BarChart;
4  import javafx.scene.chart.CategoryAxis;
5  import javafx.scene.chart.NumberAxis;
6  import javafx.scene.chart.XYChart;
7  import javafx.scene.layout.FlowPane;
8  import javafx.stage.Stage;
9
10 public class EsempioBarChart extends Application {
11     @Override
12     public void start(Stage stage) {
13         stage.setTitle("BarChart esempio");
14
15         // 1. Assi
16         CategoryAxis xAxis = new CategoryAxis();
17         xAxis.setLabel("Tipi di frutta");
18         NumberAxis yAxis = new NumberAxis();
19         yAxis.setLabel("Vendite");
20
21         // 2. Grafico
22         BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
23         chart.setTitle("Andamento vendite frutta");
24
25         // 3. Serie
26         XYChart.Series<String, Number> modena = new XYChart.Series<>();
27         modena.setName("Modena");
28
29         // 4. Popolamento
30         modena.getData().add(new XYChart.Data<>("Mele", 30));
31         modena.getData().add(new XYChart.Data<>("Pere", 15));
32         modena.getData().add(new XYChart.Data<>("Arance", 50));
33
34         // 5. Aggiunta al grafico
35         chart.getData().add(modena);
36
37         FlowPane root = new FlowPane(chart);
38         Scene scene = new Scene(root, 600, 400);
39         stage.setScene(scene);
40         stage.show();
41     }
42 }
```

Nota: i tipi generici `BarChart<String, Number>` e `XYChart.Series<String, Number>` devono corrispondere al tipo dell'asse delle categorie (String) e a quello dell'asse dei valori (Number).

Componente BubbleChart

Grafico a bolle: ogni dato ha coordinate x, y (entrambe numeriche) e la bolla ha una dimensione opzionale (di solito si usa un terzo valore). In JavaFX base, `BubbleChart` richiede due `NumberAxis` e i dati sono `XYChart.Data<Number, Number>`.

Esempio – parabola e retta

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.scene.chart.BubbleChart;
4  import javafx.scene.chart.NumberAxis;
5  import javafx.scene.chart.XYChart;
6  import javafx.scene.layout.FlowPane;
7  import javafx.stage.Stage;
8
9  public class EsempioBubbleChart extends Application {
10     @Override
11     public void start(Stage stage) {
12         NumberAxis xAxis = new NumberAxis();
13         xAxis.setLabel("x");
14         NumberAxis yAxis = new NumberAxis();
15         yAxis.setLabel("y");
16
17         BubbleChart<Number, Number> chart = new BubbleChart<>(xAxis, yAxis);
18         chart.setTitle("Funzioni varie");
19
20         // Serie parabola (y = x^2 / 10 approssimata)
21         XYChart.Series<Number, Number> parabola = new XYChart.Series<>();
22         parabola.setName("parabola");
23         parabola.getData().addAll(
24             new XYChart.Data<>(0, 0),
25             new XYChart.Data<>(10, 10),
26             new XYChart.Data<>(15, 22),
27             new XYChart.Data<>(20, 40),
28             new XYChart.Data<>(30, 90)
29         );
30
31         // Serie retta a 45°
32         XYChart.Series<Number, Number> retta = new XYChart.Series<>();
33         retta.setName("retta 45");
34         retta.getData().addAll(
35             new XYChart.Data<>(0, 0),
36             new XYChart.Data<>(5, 5),
37             new XYChart.Data<>(15, 15),
38             new XYChart.Data<>(20, 20),
39             new XYChart.Data<>(30, 30),
40             new XYChart.Data<>(40, 40),
41             new XYChart.Data<>(50, 50)
42         );
43
44         chart.getData().addAll(parabola, retta);
45
46         FlowPane root = new FlowPane(chart);
47         Scene scene = new Scene(root, 700, 500);
48         stage.setScene(scene);
49         stage.show();
50     }
51 }
```

Attenzione: il costruttore di `BubbleChart` si aspetta due `NumberAxis`. La dimensione della bolla può essere controllata con `setExtraValue()` (non mostrato qui).

Componente PieChart (grafico a torta)

Non usa assi. I dati sono una `ObservableList<PieChart.Data>`.

Passi

1. Creare la lista di `PieChart.Data` (etichetta, valore).
2. Creare il `PieChart` passando la lista nel costruttore.
3. (Opzionale) impostare titolo e animazione.

Nota: `PieChart.Data` non è tipizzato; i valori sono `double`.

Animazione dei grafici (aggiornamento dinamico)

Tutti i grafici supportano l'animazione automatica quando i dati cambiano. Basta abilitarla con `setAnimated(true)` (di solito è già `true` di default, ma è meglio impostarlo esplicitamente).

Esempio di aggiornamento su BarChart

```
1 // Supponendo di avere una serie 'ferrara' già aggiunta al grafico
2 chart.setAnimated(true);
3
4 Button aggiorna = new Button("Aggiorna Ferrara");
5 aggiorna.setOnAction(event -> {
6     // Modifica i valori Y della serie
7     ferrara.getData().get(0).setYValue(45); // Mele
8     ferrara.getData().get(1).setYValue(5);  // Pere
9     ferrara.getData().get(2).setYValue(69); // Arance
10 });
```

Importante: per i `PieChart`, la lista `dati` deve essere quella originale passata al costruttore del `PieChart`. Modificando i valori con `setPieValue()`, il grafico si aggiorna con animazione.

Tabella riassuntiva componenti più usati

Componente	Costruttore tipico	Evento principale	Lettura valore	Scrittura
Label	<code>new Label("testo")</code>	nessuno	<code>getText()</code>	<code>setText</code>
Button	<code>new Button("testo")</code>	<code>setOnAction</code>	–	–
TextField	<code>new TextField()</code>	<code>setOnAction</code> (INVIO)	<code>getText()</code>	<code>setText</code>
TextArea	<code>new TextArea()</code>	–	<code>getText()</code>	<code>setText</code>
ComboBox	<code>new ComboBox<>()</code>	<code>setOnAction</code>	<code>getValue()</code>	<code>setValu</code>
Slider	<code>new Slider(min,max,val)</code>	<code>valueProperty().addListener</code>	<code>getValue()</code>	<code>setValu</code>
ToggleButton	<code>new ToggleButton("testo")</code>	<code>setOnAction</code>	<code>isSelected()</code>	<code>setSele</code>
CheckBox	<code>new CheckBox("testo")</code>	<code>setOnAction</code>	<code>isSelected()</code>	<code>setSele</code>
RadioButton	<code>new RadioButton("testo")</code>	via <code>ToggleGroup</code>	<code>getSelectedToggle()</code>	<code>setSele</code>
ListView	<code>new ListView<>()</code>	<code>selectedItemProperty().addListener</code>	<code>getSelectionModel().getSelectedItem()</code>	<code>setItem</code>
ChoiceBox	<code>new ChoiceBox<>()</code>	<code>setOnAction</code>	<code>getValue()</code>	<code>setValu</code>
Spinner	<code>new Spinner<>(min,max,init)</code>	<code>valueProperty().addListener</code>	<code>getValue()</code>	<code>getValu</code>
ColorPicker	<code>new ColorPicker()</code>	<code>setOnAction</code>	<code>getValue()</code>	<code>setValu</code>
DatePicker	<code>new DatePicker()</code>	<code>setOnAction</code>	<code>getValue()</code>	<code>setValu</code>
ProgressBar	<code>new ProgressBar(val)</code>	<code>progressProperty().addListener</code>	<code>getProgress()</code>	<code>setProg</code>